How LLMs Process Information

A Geometric View Inside Transformer Computation

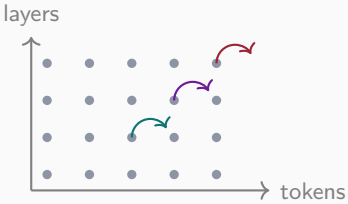
Agus Sudjianto

Based on *Learning Geometric Algebra Through Transformer Geometry*

https://github.com/asudjianto-xml/layer-time-geometry

Roadmap

1. **The Standard Picture** 8 min
   Tokens, layers, hidden states
2. **Layers as Rotations** 12 min
   Versors, bivectors, the metric as gauge
3. **When Order Matters** 8 min
   Commutators, curvature, capacity
4. **The Decode Frontier** 14 min
   Generation, directed flow, BCA
5. **Seeing Inside & Steering** 12 min
   Hallucination, monitoring, intervention
6. **Q&A** 6 min



The layer–token grid

The Standard Picture

What happens when you prompt an LLM?

- Each token becomes a **hidden-state vector** $H^{(l,t)} \in \mathbb{R}^{3584}$
- $L = 28$ layers, T tokens $\Rightarrow$ a grid of $(28 \times T)$ vectors
- Standard view: "matrix multiplies and attention." True but uninformative.

What is each layer actually doing to these vectors?

The layer–token grid

Every forward pass produces a **grid** of hidden states:

$$H : (l, t) \longmapsto \mathbb{R}^p \quad (p = 3584 \text{ for Qwen2.5-7B})$$

- Vertical axis: **layers** ($L = 28$)
- Horizontal axis: **token positions** (T tokens)
- Every dot is a high-dimensional vector

We need a language that describes not just *how much* each layer changes the vectors, but *in which directions*.

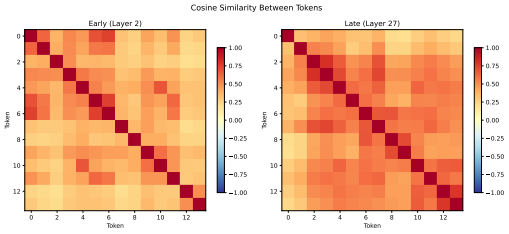


Figure: Cosine similarity between hidden states at adjacent layers. Values near 1 = small change; dips = layers that substantially redirect the vector. Each layer reshapes meaning by a different amount.

Layers as Rotations

Building an honest coordinate system

Raw hidden states live in $\mathbb{R}^{3584}$ with a **distorted** metric: distances and angles are unreliable.

Whitening (PCA + rescaling):

$$\mathbb{R}^{3584} \xrightarrow{\text{PCA}} \mathbb{R}^{256} \xrightarrow{\text{rescale}} \text{Covariance} = I_{256}$$

- Retains $> 99\%$ of variance
- Establishes an **orthonormal frame**
- Angles and rotations become *meaningful*
- Acts as a **gauge**: the "ruler" for all subsequent measurements

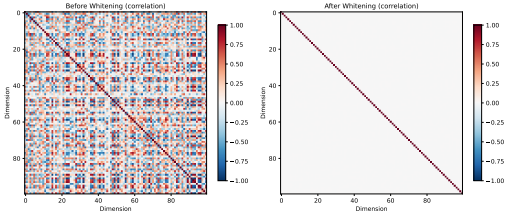


Figure: Covariance structure before and after whitening. After whitening, the covariance is the identity — distances and angles become trustworthy.

What a layer does: rotation × stretch

Each layer transition (averaged across token positions) decomposes as:

$$\boxed{T^{(l)} = \underbrace{R^{(l)}}_{\text{rotation}} \cdot \underbrace{P^{(l)}}_{\text{stretch}}}$$

(Attention makes T position-dependent; we'll see per-token rotors on the next slides.)

Rotation (grade-2, rotor)

- Reorients the hidden state — moves meaning to *different directions*
- Preserves vector lengths
- Generator: bivector $B^{(l)} = \log(R^{(l)})$

Stretch (grade-0, metric factor)

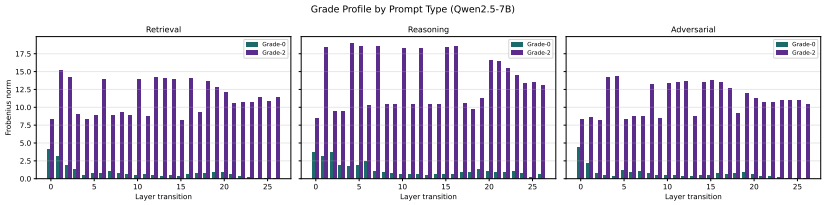
- Amplifies or suppresses specific components — *adjusts emphasis*
- Preserves directions
- Near-identity in practice

Empirical finding (Qwen2.5-7B): Rotation dominates stretch by **10–15**× in norm.

Layers mostly rotate — they redirect meaning, not rescale it.

Connection: $T^{(l)} \approx \partial H^{(l+1)}/\partial H^{(l)}$ — the versor *is* the layer Jacobian. Rotation = reorientation of meaning;

Rotation dominates: the grade profile



Purple = rotation (grade-2) norm; teal = stretch (grade-0) norm, per layer and prompt type. Purple towers over teal at every layer — rotation outweighs stretch by **10–15**×. Layers primarily redirect meaning, not rescale it.

Bivectors: the plane of rotation

A rotation in $\mathbb{R}^{256}$ doesn't have an "axis." It has a **plane**.

The bivector $B^{(l)} = \log(R^{(l)})$ is a **skew-symmetric matrix** encoding:

- **Which plane(s)** the layer rotates in
- **How much** (the angle θ)

Principal plane decomposition: $B^{(l)} = \sum_{j=1}^{128} \sigma_j \, \hat{B}_j$.

Typically **3–5 dominant planes** per layer.

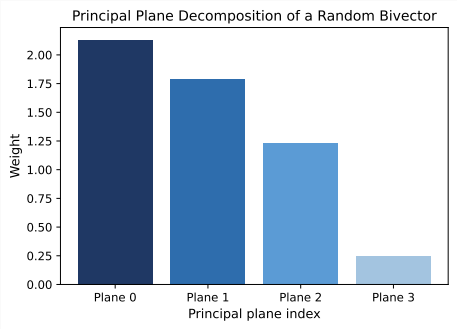


Figure: Singular values of the bivector. The first few bars dominate — rotation is concentrated in 3–5 planes, not all 128.

A layer is a field of rotations

Attention makes the transformation **position-dependent**:

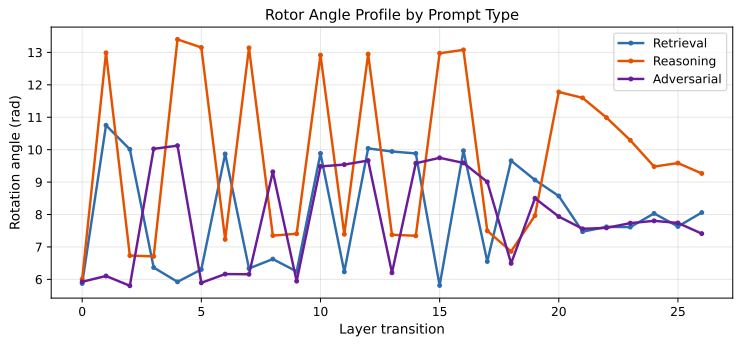
$$H^{(l+1)}(t) \;=\; R^{(l)}(t)\, P^{(l)}(t)\;\; H^{(l)}(t)$$

Each token position gets its **own rotor**.

Different rotation at each position

A layer is not "a rotation" — it is a **field of rotors**, each rotating the hidden state in its own plane by its own angle.

Rotor angles across layers



Reading the figure: Each curve traces rotation angle $\tau^{(l)}$ (degrees) per layer for one prompt type. Higher = bigger rotation. **Takeaway:** Early layers rotate aggressively (40–60°); deeper layers make finer adjustments. Different tasks produce *different* profiles — the model adapts its rotation schedule to the input.

When Order Matters

Do layers commute?

The **commutator** asks: does it matter which order I apply two layers' rotations?

$$[B^{(l)},\ B^{(l+1)}] \ = \ B^{(l)}B^{(l+1)} - B^{(l+1)}B^{(l)}$$

- $= 0$: layers are **independent** — swapping them wouldn't change the result
- $\neq 0$: layer $l{+}1$ **builds on** layer l's rotation — sequential processing matters
- Non-zero commutator = *genuinely sequential* reasoning, not parallel operations

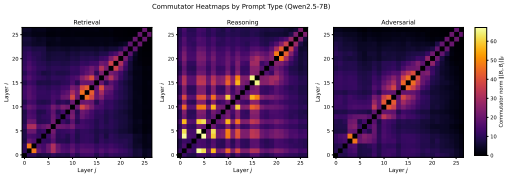


Figure: Commutator norm $\|[B^{(l)}, B^{(l+1)}]\|_F$ per layer pair. Higher = stronger sequential coupling.

Capacity: the model's non-commutative budget

Capacity = total sequential computation. Sum commutator norms over all adjacent layer pairs: $C_{\text{acc}} = \sum_{\text{adj. pairs}} \|[B^{(l)}, B^{(l+1)}]\|_F$

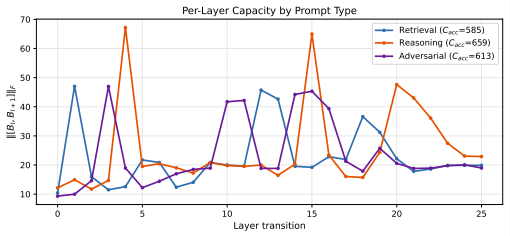


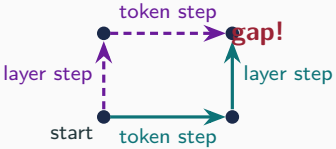
Figure: Cumulative capacity C_{acc} vs. layer. Steeper climb = more sequential computation invested up to that depth.

Prompt	C_{acc}	**Peak**
Retrieval	584.9	Layer 1
Reasoning	658.5	Layer 4
Adversarial	613.4	Layer 3

- Reasoning demands **13% more** capacity — more sequential steps needed
- Peak layer shifts with task: the model allocates its "budget" **adaptively**

Walking in circles: holonomy and curvature

Transport a vector around a closed loop on the grid. If it comes back *changed*, the space is curved.



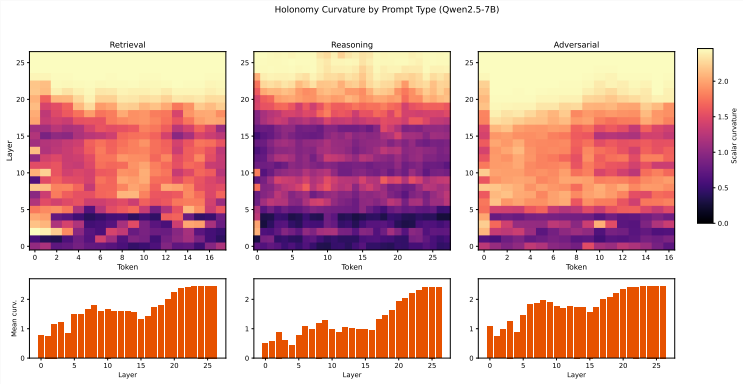
Compose all four rotors around the cell — the **loop rotor**:

$$R_{\text{loop}} = R_{\uparrow} R_{\rightarrow} R_{\uparrow}^{-1} R_{\rightarrow}^{-1}$$

If $R_{\text{loop}} \neq I$: the grid is **curved**. $\Omega = \log(R_{\text{loop}})$ tells you *which plane*.

Intuition: how a layer transforms a token *depends on which other tokens it has seen* — layer and context are entangled.

Curvature across the grid

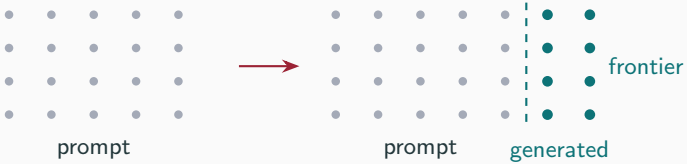


Reading the figure: Each heatmap cell is one plaquette (l, t). Color intensity = holonomy deficit $\|R_{\text{loop}} - I\|_F$. Brighter = more curvature. **Takeaway:** Non-zero curvature means layer and token processing are **entangled**. Reasoning prompts show the strongest curvature.

The Decode Frontier

From static analysis to generation

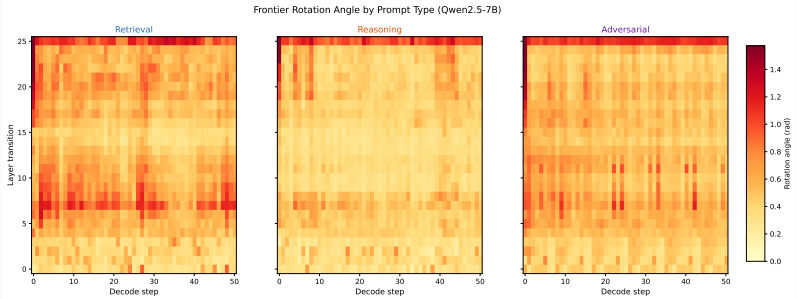
Everything so far: one prompt, one forward pass. But LLMs **generate** — one token at a time.



Each new token adds a **column** to the grid. The **decode frontier** is the evolving right edge.

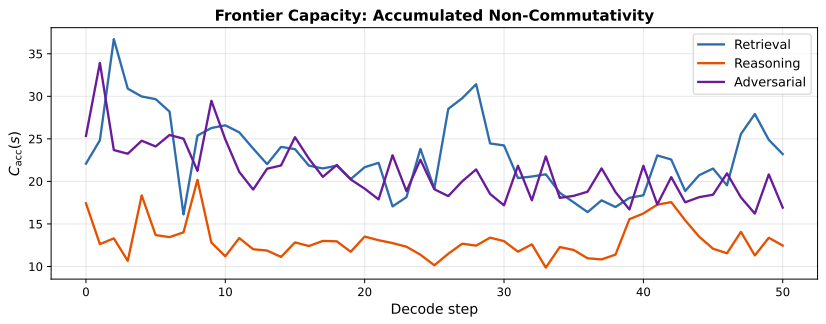
Question: How does the geometry change as the model generates?

Rotation planes drift during generation



Rotation angle at the newest generated token vs. decode step; each line is one layer. Rotation planes **drift** as context grows. Adversarial prompts show periodic cycling that mirrors stuck token loops.

Frontier capacity decays during generation



Frontier capacity C_{acc} vs. decode step for three prompt types. Capacity decays during generation. Retrieval stays high and stable; adversarial declines into periodic cycling, suggesting the model is running low on compositional resources.

PCA sees spread; BCA sees direction

The frontier is an **ordered sequence**. PCA (symmetric covariance) cannot distinguish "step 5 $\to$ step 10" from "step 10 $\to$ step 5."

$$M_\tau = \underbrace{\tfrac{1}{2}(M_\tau + M_\tau^\top)}_{C_\tau \text{ (symmetric — PCA)}} + \underbrace{\tfrac{1}{2}(M_\tau - M_\tau^\top)}_{B_\tau \text{ (antisymmetric — BCA)}}$$

Symmetric C_τ (PCA)

- Finds directions of max *spread*
- Blind to temporal order
- Treats forward = backward

Antisymmetric B_τ (BCA)

- Finds planes of max *directed flow*
- B_τ is a **bivector**
- Reverses sign if time runs backward

PCA sees where the frontier is. BCA sees where the frontier goes.

BCA detects the repetition cycle

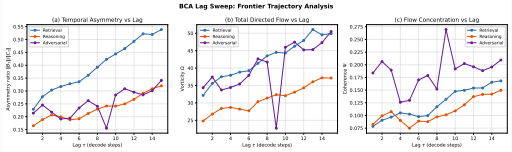


Figure: BCA lag sweep. At lag 9 (the repetition cycle), Adversarial shows a sharp drop in directed flow and spike in coherence.

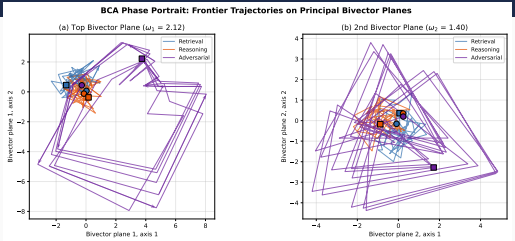


Figure: BCA phase portrait. Trajectories projected onto the top two *directed-flow* planes. Adversarial (purple) traces a large structured orbit.

Three diagnostics from B_τ:

Directed flow Ω

Total temporal asymmetry

Coherence $\Psi = \omega_1/\Omega$

Flow concentration in one plane

Asymmetry ratio $\|B\|/\|C\|$

Directional vs. undirected

Statistical caveat: with ∼60 decode steps and k=256, the antisymmetric component B_τ can be noisy. The diagnostics above are computed from whitened, centred trajectories and averaged across layers to mitigate finite-sample bias.

Hallucination as geometric collapse

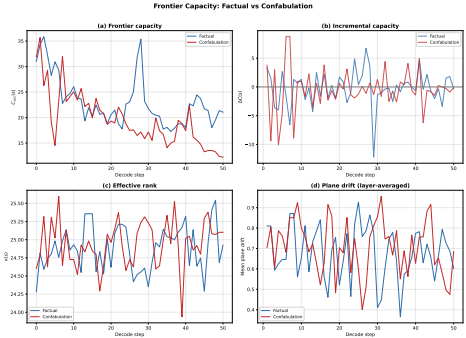


Figure: Two capacity-vs-step curves: factual (blue) vs. confabulated (red). Steeper slope = faster loss of sequential reasoning budget.

Comparing factual vs. confabulated generation:

	Factual	**Confab.**
Capacity decay	-0.18	-0.28
Curvature accel.	0.186	0.221

- **56% faster** capacity decay
- Curvature not just higher but **accelerating**
- Model loses geometric structure as it fabricates

In this model, hallucination has a measurable geometric signature *during* generation.

Seeing Inside and Steering

Online monitoring: catching failure in real time

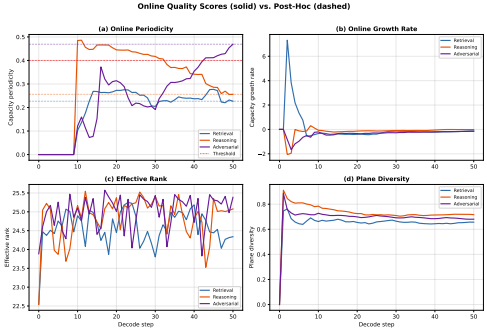


Figure: Online quality scores during generation. The Adversarial prompt crosses the alert threshold at step 42 — **before** human-visible repetition begins.

Three scores computed at each decode step:

1. **Capacity growth rate** — is the model's sequential budget draining too fast?
2. **Plane diversity** — are rotation planes collapsing into one?
3. **Curvature acceleration** — is the geometry becoming unstable?

Empirical result (Qwen2.5-7B): GA-based monitoring detects degradation $\sim$**8 steps before** token-level repetition becomes visible in our experiments.

Which layers matter?

Dependency asks: if I perturbed this layer's output, how much would the prediction change? Measured via gradient norm: $D_l = \|\partial s / \partial H^{(l)}\|_F$.

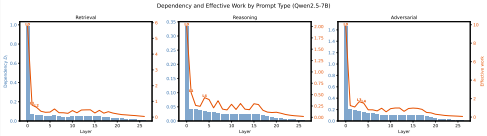


Figure: Each bar = gradient-based dependency D_l per layer. Taller = the output is more sensitive to that layer's contribution.

Effective work combines "how much the layer rotates" with "how much the output cares": $W_l = D_l \cdot \theta^{(l)}$

- The **loudest** layer (biggest rotation) is often *not* the most important
- The small stretches (grade-0) sit at high-dependency layers — geometrically quiet but **functionally decisive**

Effective work pinpoints where the model is both *active* and *consequential*.

Cayley steering: from detection to control

The Cayley bivector between *query-only* and *context-augmented* hidden states tells us **where** and **in which plane** context acts:

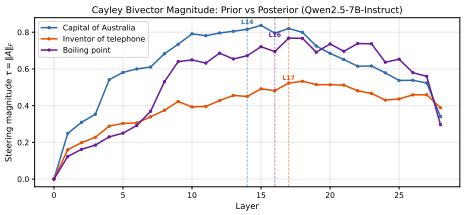


Figure: $\tau^{(l)}$ per layer for three counterfactual prompts. The peak identifies where context most strongly rotates the hidden state.

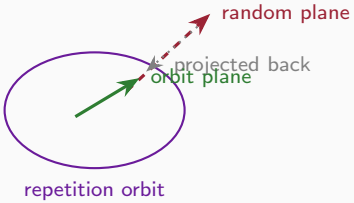
Prompt	Layer	τ	Output
Capital of Australia	14	0.84	Sydney
Inventor of telephone	17	0.53	Tesla
Boiling point	16	0.77	250°C

The **steering recipe**:

1. Compute $\tau^{(l)}$ — find the peak layer
2. Extract the principal plane (v_1, v_2) of the Cayley bivector
3. Apply a Rodrigues rotation by angle α in that plane during generation

In this example, one GA object — the Cayley bivector — provides detection, diagnosis, and targeted intervention.

Why the plane matters: random steering fails



In k=256 dimensions, a random 2D plane is overwhelmingly unlikely to overlap with the orbit's specific 2D subspace.

Experiment: apply random-plane rotations at the frontier to break the adversarial repetition cycle.

- Even at $\alpha = 0.20$ rad: **orbit persists**
- The model's nonlinear layers project the perturbation **back onto the attractor**
- Steering must target the right plane — the one the model actually uses

This negative result motivates the Cayley steering recipe: use the model's *own* rotation planes.

When context fails: steering past parametric resistance

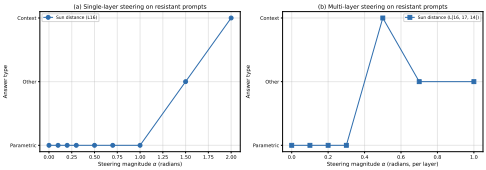


Figure: Steering sweep on a resistant prompt. (a) Single-layer: requires α=2.0. (b) Multi-layer: context answer at α=0.5 with a narrow effective window.

Context says "Earth–Sun distance = **50 million miles**." Model ignores it, answers "93 million miles."

The Cayley bivector *detects* context influence ($\tau = 0.70$ at layer 16) — the hidden states are rotated, but not enough to flip the output.

Steering	α	**Result**
None	—	93 million (parametric)
Single L16	2.0	1 million (context)
Multi L16,17,14	**0.5**	**50 million (exact)**

Takeaway (single example): Multi-layer Cayley steering at moderate α can override parametric resistance that context alone cannot — a geometrically targeted nudge in the model's own rotation planes.

The Big Picture

A transformer is a geometric field system

The entire framework is captured in **three fields** over a discrete grid:

$$
\begin{aligned}
H : \mathcal{G} &\longrightarrow \mathbb{R}^k && \text{(hidden-state field)} \\
R : \mathcal{G} &\longrightarrow \mathrm{Spin}(k) && \text{(rotor field)} \\
\Omega : \mathcal{G} &\longrightarrow \mathfrak{so}(k) && \text{(curvature field)}
\end{aligned}
$$

$$\textit{Vectors} \longrightarrow \textit{Rotations} \longrightarrow \textit{Curvature}$$

What GA reveals that norms cannot

Scalar summaries tell you **how much**. GA tells you **how much** and **in which direction**.

Static (single forward pass):

1. Which planes each layer rotates in
2. Which planes the curvature acts in
3. Which planes layers fail to commute in
4. Which planes the output depends on

Dynamic (during generation):

5. How rotation planes drift or collapse
6. **Directed flow** via BCA (temporal order)
7. Whether capacity decay signals hallucination
8. Real-time monitoring *before* failure

A transformer is a discrete geometric field system whose computation is governed by rotations and curvature.

Summary

Concept	Intuition	GA object
Versor decomp.	Layer rotates vs. stretches	$T = R \cdot P$
Bivector field	Which directions reshuffled	$B^{(l)} \in \mathfrak{so}(k)$
Commutator	Does layer order matter?	$[B^{(l)}, B^{(l+1)}]$
Capacity	Sequential computation budget	$C_{\text{acc}} = \sum \|[\cdot]\|_F$
Holonomy	Layer–context entanglement	R_{loop}
Dependency	Which layers matter	$D_l = \|\partial s / \partial H^{(l)}\|_F$
BCA	Directed temporal flow	$B_\tau = (M - M^\top)/2$
Monitoring	Real-time failure detection	Quality scores at frontier

Key takeaways:

1. Layers are predominantly **rotations**; non-commutativity underlies sequential reasoning
2. **Curvature** reveals layer–context entanglement
3. **BCA** recovers temporal direction that PCA discards
4. Hallucination has a measurable **geometric signature** (in our experiments)
5. The Cayley bivector enables **plane-specific intervention**

Questions?

How LLMs Process Information: A Geometric View

Agus Sudjianto

Based on *Learning Geometric Algebra Through Transformer Geometry*

`https://github.com/asudjianto-xml/layer-time-geometry`

Model: Qwen2.5-7B (28 layers, whitened to $k = 256$) · Library: `layer_time_ga`